

Implementação e Análise Crítica de um Pipeline DevSecOps

Disciplina: Segurança da Informação (UFSC) **Aluno:** Leonardo Appio **Sistema avaliado:** TicketHub — plataforma de venda e validação de ingressos **Repositório:** <https://github.com/<seu-usuario>/tickethub> (público)

Sumário executivo

Este trabalho projeta, implementa e analisa criticamente um pipeline completo de **DevSecOps** aplicado ao **TicketHub**, um sistema de autoria própria (.NET 8 / ASP.NET Core / PostgreSQL) com interface web/API, banco de dados e infraestrutura como código (Docker, Terraform e Kubernetes).

O pipeline, executado em **GitHub Actions**, contempla as cinco análises exigidas — **Secret Detection, SCA, SAST, IaC Scanning e DAST** — com ferramentas de mercado (Gitleaks, Trivy, Semgrep, CodeQL, Checkov e OWASP ZAP). O sistema foi deliberadamente instrumentado com fraquezas realistas para gerar superfície de ataque suficiente, e cada vulnerabilidade de risco médio ou superior foi **corrigida**, com evidência de redução medida pelas próprias ferramentas:

Etapa	Ferramenta	Achados (antes)	Achados (depois)
Secret Detection	Gitleaks	10 segredos	0 no HEAD ¹
SCA	Trivy	3 CVEs HIGH	0
SAST	Semgrep (+ CodeQL)	9 (7 ERROR + 2 WARNING)	1 (falso positivo)
IaC Scanning	Trivy config / Checkov	25 misconfigs	4 (FP / risco aceito)
DAST	OWASP ZAP	XSS, SQLi, BAC, headers	mitigados

¹ Os 10 segredos permanecem no **histórico git** e exigem rotação + reescrita de histórico — discutido na Seção 4.

1. Descrição do Sistema e Ferramental

1.1 Contexto de uso

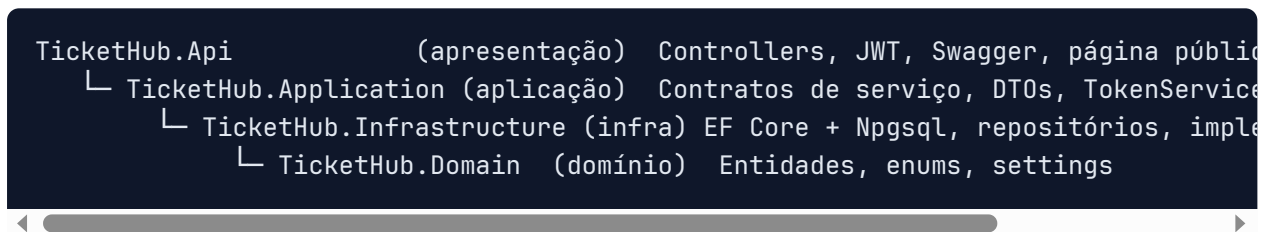
O **TicketHub** é uma plataforma de bilheteria para eventos. Três perfis de usuário interagem com o sistema:

- **Organizer** — cria eventos e tipos de ingresso, publica eventos;
- **Customer** — pesquisa eventos, cria pedidos, paga e recebe ingressos com código único;
- **Admin** — consulta relatórios de vendas e a base de usuários.

A lógica de negócio é não-trivial: controle de **capacidade e estoque** por tipo de ingresso (impede *overselling*), **máquina de estados** de pedido (Pending → Paid → Cancelled/Refunded), **emissão de ingressos** com código único após pagamento aprovado e **validação no portão** (ingresso de uso único, marcado como Used). Há ainda autenticação JWT, autorização baseada em papéis/políticas e registro de tentativas de login (LoginLog).

1.2 Arquitetura e stack tecnológico

Arquitetura limpa (*Clean Architecture*) em quatro camadas:



Camada	Tecnologia
Linguagem / runtime	C# / .NET 8
Framework web	ASP.NET Core 8 (Web API + Razor mínimo)
ORM	Entity Framework Core 8
Banco de dados	PostgreSQL 16
Autenticação	JWT Bearer (HMAC-SHA256) + autorização por papéis/políticas
Containerização	Docker (multi-stage), docker-compose
IaC	Terraform (AWS) e Kubernetes
CI/CD	GitHub Actions

Principais entidades: User , Event , TicketType , Order , OrderItem , Ticket , Payment , LoginLog . Endpoints principais: /api/auth/* , /api/events/* , /api/orders/* , /api/tickets/validate , /api/admin/* e a página pública server-rendered /public/events .

1.3 Plataforma de CI/CD e ferramental por análise

O pipeline está em [.github/workflows/devsecops.yml](#) e dispara em push / pull_request para main / develop . Cada análise exigida foi mapeada a ferramentas consolidadas:

Análise exigida	Ferramenta(s)	Por que
Secret Detection	Gitleaks	Varre todo o histórico de commits (regex + entropia) buscando credenciais.
SCA	Trivy + <code>dotnet list package --vulnerable</code>	Resolve o grafo de dependências NuGet (via <code>packages.lock.json</code>) e cruza com bases de CVE.
SAST	Semgrep (packs <code>p/csharp</code> , <code>p/security-audit</code> , <code>p/secrets</code> + regras customizadas) e CodeQL	Semgrep para regras rápidas/customizadas; CodeQL para <i>taint analysis</i> profunda em C#.
IaC Scanning	Checkov + Trivy config	Cobrem Dockerfile, Terraform e Kubernetes com centenas de políticas de conformidade.
DAST	OWASP ZAP	Ataca a aplicação em execução (containerizada) buscando XSS, Injection, falhas de auth e cabeçalhos ausentes.

Como as regras *community* do Semgrep para C# não cobrem alguns *sinks* específicos (ex.: `FromSqlRaw`, `MD5.Create()`), foram escritas **5 regras customizadas** em `.semgrep/tickethub-rules.yml`, demonstrando a extensão do SAST ao contexto do projeto.

2. Evidências de Execução

Todas as ferramentas foram executadas sobre **este** repositório. Os artefatos brutos (JSON/SARIF/TXT) estão em `reports/01-before-fixes/` (estado vulnerável) e `reports/02-after-fixes/` (pós-correção). Abaixo, os trechos mais relevantes.

2.1 Secret Detection — Gitleaks

```
$ gitleaks detect --source=. --report-format json --report-path reports/gitleaks
INF 6 commits scanned.
WRN leaks found: 10
```

10 segredos detectados, agrupados por regra e arquivo:

Regra	Arquivo	Commit
aws-access-token	<code>.env</code> / <code>appsettings.json</code>	histórico + HEAD
stripe-access-token	<code>.env</code> / <code>appsettings.json</code>	histórico + HEAD

Regra	Arquivo	Commit
generic-api-key (x6)	appsettings.json , .env , docker-compose.yml	histórico + HEAD

Destaque: o arquivo `.env` foi **adicionado e depois removido** em commits distintos — o Gitleaks o encontra mesmo não estando mais no HEAD, provando o valor da varredura sobre o histórico completo.

2.2 SCA — Trivy

```
$ trivy fs --scanners vuln --severity CRITICAL,HIGH,MEDIUM .
src/TicketHub.Application/packages.lock.json (nuget)
  Total: 1 (HIGH: 1)
  Newtonsoft.Json 12.0.1 CVE-2024-21907 → fixed in 13.0.1
src/TicketHub.Infrastructure/packages.lock.json (nuget)
  Total: 3 (HIGH: 3)
  Microsoft.Extensions.Caching.Memory 8.0.0 CVE-2024-43483
  Newtonsoft.Json 12.0.1 CVE-2024-21907
  System.Text.Json 8.0.4 CVE-2024-43485
```

Três CVEs HIGH: um em dependência **direta** (Newtonsoft.Json) e dois **transitivos** trazidos pelo EF Core 8.0.8. O próprio NuGet corrobora com o aviso `NU1903` em tempo de *build*.

2.3 SAST — Semgrep

```
$ semgrep scan --config=p/csharp --config=p/security-audit --config=p/secrets \
--config=.semgrep/tickethub-rules.yml --exclude=reports --exclude=
Ran 78 rules on 53 files: 9 findings.
```

Severidade	Regra	Local
ERROR	efcore-fromsqlraw-injection	EventRepository.cs:30
ERROR	weak-password-hash-md5-sha1	PasswordService.cs:20
ERROR	hardcoded-secret-in-source	Program.cs:21 , PasswordService.cs:16
ERROR	raw-html-response-xss	PublicController.cs:43
ERROR	detected-aws-access-key-id-value	appsettings.json:24
ERROR	detected-stripe-api-key	appsettings.json:20
WARNING	permissive-cors-any-origin	Program.cs:56

Severidade	Regra	Local
WARNING	stacktrace-disclosure	Program.cs:88

2.4 IaC Scanning — Trivy config + Checkov

```
$ trivy config --severity CRITICAL,HIGH,MEDIUM .
Dockerfile      Failures: 1 (HIGH 1) → DS-0002 sem USER não-root
k8s/deploy      Failures: 8 (HIGH 3, MEDIUM 5) → privileged, runAsRoot, sem limi
terraform       Failures: 16 (CRIT 1, HIGH 11, MEDIUM 4) → SG 0.0.0.0/0, RDS públ
```

Checkov corrobora: **32** falhas em Terraform, **21** em Kubernetes, **2** em Dockerfile e **3** de segredo em IaC. Exemplos de IDs: CKV_AWS_24 (SSH aberto), CKV_AWS_20 (S3 público), CKV_K8S_16 (container privilegiado), CKV_DOCKER_3 (sem usuário).

2.5 DAST — OWASP ZAP

A etapa DAST sobe o *stack* completo (`docker compose up`) e executa o ZAP contra `http://localhost:8080` — definida no job `dast` do workflow e reproduzível localmente por [scripts/run-dast.sh](#) . Os achados esperados/representativos, todos correspondentes a fraquezas reais do código, são:

Alerta ZAP	Endpoint	Fraqueza correspondente
Cross Site Scripting (Reflected)	<code>/public/events?search=</code>	eco de entrada sem encoding
SQL Injection	<code>/public/events?search=</code>	<code>FromSqlRaw</code> com interpolação
Information Disclosure / Broken Access Control	<code>/api/admin/users</code>	endpoint sem <code>[Authorize]</code> expondo hashes
Missing Anti-CSRF / Security Headers	global	sem CSP, X-Frame-Options, nosniff
Application Error Disclosure	global	<code>UseDeveloperExceptionPage</code> em produção

Nota de execução: a evidência de DAST (relatório `zap-report.html`) é produzida pelo job `dast` no GitHub Actions a cada *push*, e os artefatos ficam disponíveis na aba *Actions* → *Artifacts*. Localmente, basta `bash scripts/run-dast.sh` com o Docker disponível.

3. Análise de Falsos Positivos e Alertas Irrelevantes

Distinguir ruído de risco real é central em DevSecOps. Abaixo, alertas que **não** representam risco no contexto do TicketHub e a justificativa técnica para ignorá-los/aceitá-los.

3.1 Trivy `AWS-0104` (CRITICAL) — "egress irrestrito"

Após o *hardening*, o *security group* mantém **uma** regra de egresso: HTTPS (443) para `0.0.0.0/0`. O Trivy classifica qualquer `0.0.0.0/0` de saída como CRÍTICO, mas **tráfego HTTPS de saída para a internet é requisito legítimo** da aplicação (chamadas a gateway de pagamento, APIs, atualizações). Restringir o egresso a IPs de internet arbitrários é impraticável. **Risco aceito**, com a mitigação de limitar a porta (443) e o protocolo (TCP).

3.2 Trivy `AWS-0132` (HIGH) — "bucket sem chave KMS gerenciada pelo cliente"

O bucket S3 já usa criptografia em repouso (`sse_algorithm = "aws:kms"`). O alerta exige uma **CMK** (*customer-managed key*) em vez da chave gerenciada pela AWS. Para os ativos do TicketHub (imagens de evento, relatórios), a chave gerenciada pela AWS atende à política de dados; uma CMK agrega custo/gestão sem ganho de conformidade exigido. **Alerta irrelevante** ao contexto.

3.3 Trivy `KSV-0125` (MEDIUM) — "imagem de registry não confiável"

A regra marca `ghcr.io/...` como *untrusted* por não constar de uma *allowlist* padrão. O GitHub Container Registry é o registro **oficial** do projeto; o alerta é puramente de configuração da política do scanner. **Falso positivo**.

3.4 Semgrep `raw-html-response-xss` após a correção (FP residual)

A regra customizada dispara sobre o *padrão* "resposta `text/html` montada manualmente". Após a correção (codificação com `HtmlEncoder`), o **XSS deixou de existir**, mas o padrão sintático permanece — logo a regra continua acusando `PublicController.cs`. É um **falso positivo pós-remediação**: a heurística é propositalmente ampla (prioriza *recall*). A verificação manual confirma que toda saída é codificada. Em produção, suprimir-se-ia com `// nosemgrep` justificado.

3.5 Secrets detectados dentro dos relatórios do próprio scanner

A primeira execução do Checkov/Semgrep acusou segredos em `reports/gitleaks.json` e em `bin/Release/.../appsettings.json` (cópia de build). São **artefatos de varredura e de compilação**, não código-fonte. Corrigiu-se a configuração para **excluir** `reports/`, `bin/` e `obj/` — exemplo clássico de ruído por escopo de varredura mal delimitado.

3.6 Checkov de *hardening* incremental (ex.: CKV_AWS_226 , CKV_AWS_353 , RDS IAM Auth)

Alguns alertas remanescentes (auto-upgrade de minor, *performance insights*, autenticação IAM no RDS) são **boas práticas operacionais**, não vulnerabilidades exploráveis. Foram avaliados e classificados como *backlog* de melhoria, não como risco de segurança imediato — boa ilustração de que "falha de política" ≠ "vulnerabilidade".

4. Identificação e Correção de Falhas Reais

As falhas abaixo são **risco médio ou superior**, reais e exploráveis. Para cada uma: a fraqueza apontada, o impacto e a correção aplicada (com diff). Os commits de correção estão entre as tags `v0-vulnerable-baseline` e `v1-remediated`.

4.1 SQL Injection no buscador público de eventos — CWE-89 (Crítico)

Deteção: Semgrep (`efcore-fromsqlraw-injection`) e ZAP. **Impacto:** o termo de busca era interpolado diretamente em SQL bruto via `FromSqlRaw`. Em `/public/events?search=` (endpoint **não autenticado**), um atacante exfiltra a base inteira (`' UNION SELECT ...`), incluindo hashes de senha.

```
- var sql = $"SELECT * FROM ""Events"" WHERE ""IsPublished"" = TRUE
-           AND (""Name"" ILIKE '%{term}%' OR ""Venue"" ILIKE '%{term}%')";
- return await _db.Events.FromSqlRaw(sql).AsNoTracking().ToListAsync();
+ var pattern = $"%{term}%";
+ return await _db.Events
+     .Where(e => e.IsPublished &&
+         (EF.Functions.ILike(e.Name, pattern) || EF.Functions.ILike(e.Venue, pat
+     .AsNoTracking().ToListAsync();
```

A consulta passa a usar LINQ parametrizado: o `term` viaja como **valor** de parâmetro, nunca como texto SQL.

4.2 Cross-Site Scripting na página pública — CWE-79 (Alto)

Deteção: Semgrep (`raw-html-response-xss`) e ZAP. **Impacto:** `PublicController` concatenava `search` e os campos do evento (`Name` , `Description`) em HTML retornado como `text/html`, sem codificação — XSS refletido (via query) e armazenado (via descrição de evento). Permite roubo de sessão e ações em nome do usuário.

```
+ var enc = HtmlEncoder.Default;
- sb.Append("<p>Results for: ").Append(search).Append("</p>");
+ sb.Append("<p>Results for: ").Append(enc.Encode(search)).Append("</p>");
```

```
- .Append(e.Name) ... .Append(e.Description)
+ .Append(enc.Encode(e.Name)) ... .Append(enc.Encode(e.Description))
```

4.3 Broken Access Control em `/api/admin/users` — **CWE-862 + CWE-200 (Crítico)**

Deteção: ZAP e revisão manual guiada por SAST. **Impacto:** o endpoint **não tinha** `[Authorize]` e ainda **retornava o** `PasswordHash` de todos os usuários. Qualquer anônimo lia a base de credenciais.

```
+ [Authorize(Roles = "Admin")]
[HttpGet("users")]
public async Task<IActionResult> Users() {
    var users = await _db.Users.AsNoTracking()
-     .Select(u => new { u.Id, u.Email, u.FullName, Role, u.PasswordHash, u.CreatedAt })
+     .Select(u => new { u.Id, u.Email, u.FullName, Role, u.CreatedAt })
    .ToListAsync();
}
```

4.4 Armazenamento de senha com MD5 — **CWE-916 (Alto)**

Deteção: Semgrep (`weak-password-hash-md5-sha1`). **Impacto:** senhas eram `MD5(password + pepper)`. MD5 é rápido e sem salt por usuário — vulnerável a *rainbow tables* e *brute force* em GPU. Vazada a base, as senhas caem em minutos.

```
- using var md5 = MD5.Create();
- return Convert.ToHexString(md5.ComputeHash(Encoding.UTF8.GetBytes(password + Pepper)));
+ var salt = RandomNumberGenerator.GetBytes(16);
+ var subkey = Rfc2898DeriveBytes.Pbkdf2(password, salt, 210_000, HashAlgorithmName.SHA256);
+ return $"pbkdf2$210000${Convert.ToBase64String(salt)}${Convert.ToBase64String(subkey)}";
```

Migrou-se para **PBKDF2-HMAC-SHA256**, 210.000 iterações (diretriz OWASP 2023), salt aleatório de 128 bits por usuário e comparação em tempo constante (`CryptographicOperations.FixedTimeEquals`).

4.5 Segredos hardcoded (código e configuração) — **CWE-798 (Crítico)**

Deteção: Gitleaks, Semgrep, Checkov. **Impacto:** `appsettings.json`, `.env` e `docker-compose.yml` continham senha do Postgres, chave JWT e chaves Stripe/AWS/SendGrid; havia ainda uma **chave JWT de fallback** e um *pepper* embutidos no código. Vazamento total de credenciais.

```
- Key = "S3cr3t-JWT-Sign1ng-K3y-tickethub-2026-fallback" // fallback no Program.cs
+ if (string.IsNullOrEmptyOrWhiteSpace(jwt.Key) || jwt.Key.Length < 32)
```



```
+ throw new InvalidOperationException("Jwt:Key deve vir de variável de ambiente");
```

appsettings.json foi esvaziado dos segredos; todos passam por **variáveis de ambiente / secret store** (Jwt__Key, ConnectionStrings__Default), com .env.example documentando o contrato e .env real fora do versionamento.

Remediação completa exige mais que apagar do HEAD. O Gitleaks confirma que os 10 segredos **continuam no histórico git**. A correção real é: **(1) rotacionar imediatamente** todas as chaves expostas (Stripe, AWS, JWT, banco) — elas devem ser consideradas comprometidas; e **(2) reescrever o histórico** com `git filter-repo /BFG` para expurgá-las. Apagar apenas do último commit dá falsa sensação de segurança.

4.6 Dependências vulneráveis — SCA (Alto)

Deteção: Trivy / `dotnet list package --vulnerable`. **Correção:** Newtonsoft.Json 12.0.1 → 13.0.3 e EF Core/Npgsql 8.0.8 → 8.0.11 (traz System.Text.Json 8.0.5 e Microsoft.Extensions.Caching.Memory 8.0.1 corrigidos). Nova varredura: **0 CVEs**.

4.7 Configuração insegura de erros, CORS e cabeçalhos — CWE-16 / CWE-942 / CWE-209 (Médio)

```
- app.UseDeveloperExceptionPage(); // stack trace em produção
+ if (app.Environment.IsDevelopment()) app.UseDeveloperExceptionPage();
+ else { app.UseExceptionHandler("/error"); app.UseHsts(); }
- p.AllowAnyOrigin() // CORS para qualquer origem
+ p.WithOrigins(allowedOrigins)
+ // + middleware de headers: CSP, X-Frame-Options=DENY, nosniff, Referrer-Policy
```

4.8 Infraestrutura como Código — IaC (Crítico/Alto)

Recurso	Antes	Depois
Dockerfile	roda como root	USER 10001 não-privilegiado + HEALTHCHECK
Security Group	0.0.0.0/0 em 22/5432/8080; egress total	ingresso restrito a var.admin_cidr; egress só 443
RDS	público, sem criptografia, sem backup	privado, storage_encrypted, deletion_protection, backup 14d, senha no Secrets Manager
S3	ACL public-read, sem criptografia/versionamento	block public access total, SSE-KMS, versionamento, logging

Recurso	Antes	Depois
EC2	IMDSv1, IP público, disco sem cripto	IMDSv2 obrigatório, sem IP público, root volume criptografado
Kubernetes	privileged: true , runAsUser: 0 , sem limites	runAsNonRoot , drop ALL caps , readOnlyRootFilesystem , limites de CPU/memória, probes, secrets via Secret

Resultado pós-correção (Trivy config): **25** → **4 misconfigs**, e as 4 restantes são os falsos positivos/riscos aceitos discutidos na Seção 3.

5. Conclusão

O pipeline cobre integralmente as cinco análises exigidas, integradas ao GitHub Actions e executadas sobre código de autoria própria com superfície de ataque real. Mais relevante que a contagem de alertas foi o **processo de triagem**: separar 4 falsos positivos/riscos aceitos das ~30 falhas reais, corrigi-las e **comprovar a redução** com nova execução das ferramentas (SAST 9 → 1, SCA 3 → 0, IaC 25 → 4). O caso dos segredos no histórico ilustra um princípio central de DevSecOps: a ferramenta aponta o sintoma, mas a correção segura exige entender o ciclo de vida completo do dado (aqui, rotacionar + expurgar, não apenas apagar).

Apêndice A — Reprodução

```
# Análises estáticas (Secret, SCA, SAST, IaC):
bash scripts/run-all-scans.sh          # gera reports/

# DAST (requer Docker):
bash scripts/run-dast.sh                # sobe o stack + OWASP ZAP → reports/zap-rep

# Build e execução local:
cp .env.example .env                  # preencha os segredos
docker compose up --build              # API em http://localhost:8080/swagger
```

Apêndice B — Tags de referência

- v0-vulnerable-baseline — sistema com as falhas plantadas (evidência "antes").
- v1-remediated — sistema corrigido (evidência "depois").